

Lab 3 — Architecture Selection & UML Component Diagram

System: Student Project Portfolio & Showcase App

Architecture selection: We chose Layered Architecture for the Student Project Portfolio & Showcase App.

Why this architecture fits: The app has clear responsibilities: user interaction, project/submission management, judging and rubric scoring, artifact storage, voting protection, and public showcase discovery. Separating these responsibilities into components makes the system easier to understand, test, and maintain. The UI communicates with application services rather than accessing storage directly.

Scenario-specific reasons: 1. Student teams need to submit project metadata and multimedia artifacts, while judges need to score projects using digital rubrics. Separating portfolio management from judging keeps submission workflows distinct from scoring rules. 2. Public visitors need to browse showcases and vote, so catalog/search and voting are separate components; this lets the system apply voting checks without slowing or complicating ordinary browsing.

Security advantage: A layered design creates controlled service boundaries. Upload validation and authorization can be enforced in the project/artifact path; only authorized judges can submit scores; and the Voting & Anti-Sybil Service can validate eligibility, rate-limit requests, and detect duplicate or suspicious voting. The UI does not directly modify stored records.

Performance benefit: Separating showcase search from voting and judging allows common browse requests to be handled without invoking the scoring or vote-validation workflow. Media can be stored and retrieved through a dedicated artifact component, and frequently requested project metadata can be cached at the catalog layer if needed.

Functional requirements: • Students can create teams/projects and submit project descriptions, links, images, videos, or other approved artifacts. • Judges can evaluate projects using digital rubrics and record scores/feedback. • Visitors can browse and search published showcases and cast public votes. • The system tracks vote status and applies anti-sybil protections.

Non-functional requirements: • Security: authentication/authorization for student and judge actions, upload validation, privacy controls, and abuse-resistant voting. • Performance: responsive project browsing and predictable submission/scoring response times. • Usability: clear submission steps, accessible showcase pages, and understandable rubric/voting flows. • Reliability: preserve submissions and scores and provide useful error messages.

Main challenges and mitigations: Performance: large media and peak voting traffic; use optimized media storage, pagination/caching, and rate limits. Usability: different user roles have different tasks; provide role-appropriate screens and clear status feedback. Security: malicious uploads, unauthorized scoring, and vote manipulation; validate files and inputs, enforce least privilege, log important actions, and apply anti-sybil checks.

Component and interface summary: Six components are shown: Student/Judge/Visitor UI; Portfolio & Project Manager; Judging & Rubric Service; Artifact/Media Storage; Voting & Anti-Sybil Service; Showcase Catalog & Search. Interfaces include IProjectAPI, IRubricAPI, IArtifactStore, IVoteAPI, IShowcaseData, and IScoreData. Provided/required labels are indicated on the diagram; verify connector direction and interface notation against your instructor's UML conventions before submission.

Other components besides Order Manager and Payment Service: For this scenario, three suitable components are Portfolio & Project Manager, Judging & Rubric Service, and Voting & Anti-Sybil Service. Artifact/Media Storage and Showcase Catalog & Search are additional useful components.